

P1 · FUNÇÕES (envolvem o valor)	
<code>int("123")</code> → 123	texto → inteiro; mata zero à esq. (<code>int("02")</code> → 2)
<code>float("12.5")</code> → 12.5	texto → decimal (ponto, NUNCA vírgula)
<code>str(42)</code> → "42"	número → texto
<code>input("msg")</code>	lê do usuário; retorna SEMPRE str
<code>print(x, y)</code>	mostra; vírgula põe espaço entre args
<code>len(x)</code>	tamanho: lista, string, df (nº de linhas)
<code>round(x, 2)</code>	arredonda em 2 casas
<code>sum(l) / max(l) / min(l)</code>	soma / maior / menor de lista
<code>range(5)</code>	0,1,2,3,4 (pra <code>for i in range(n)</code>)
<code>type(x)</code>	debug: qual o tipo disso?
TEU ERRO Nº1 (5x): função RETORNA, não altera. <code>int(x)</code> solto joga fora. SEMPRE <code>x = int(x)</code> . Depois de <code>TODO input/split</code> : "converto pra quê? atribuí?". Comparar/contar com <code>str = TypeError</code> ou <code>False</code> eterno (<code>"1" == 1</code> é <code>False</code>).	
⚠ Preço em <code>input = float()</code>, nunca <code>int()</code> (centavos quebram <code>int("10.5")</code>).	
P2 · STRINGS (.depois do valor)	
<code>s.split(" ")</code>	corta → LISTA de str (converter números!)
<code>s.upper() / s.lower()</code>	MAIÚSCULA / minúscula (atribuir!)
<code>s.strip()</code>	tira espaços das pontas
<code>s.isdigit()</code>	True só se TUDO é dígito ("12.3", "-5", "12" → <code>False</code>)
<code>s.replace("a", "b")</code>	troca todas as ocorrências
<code>s[0:4]</code>	fatia: posições 0,1,2,3 (fim NÃO entra; conta do 0)
<code>"x" in s / x in lista</code>	contém?
<code>a, b, c = s.split("/")</code>	desempacota: 1 nome por pedaço (nº igual!)
<code>partes[0]</code>	caixinha 0 da lista (= coluna A do Texto p/ Colunas)

⚠ Dois separadores num `split` NÃO existe: `split` em ESTÁGIOS (espaço primeiro separa os mundos, depois / abre a data). Campo opcional: só acessar `partes[1]` DENTRO de `if len(partes)==2:` .

P3 · F-STRING (montar texto)	
<code>f"Oi {nome}"</code>	<code>f</code> liga o modo; <code>{}</code> injeta variável
<code>f"{x:.2f}" / f"{x:.1f}"</code>	2 casas / 1 casa decimal
UMA string contínua. Sem <code>+</code> , sem <code>{}</code> fora do <code>f""</code> . "Paciente" x (justaposição) = <code>SyntaxError</code> . Escreve a frase literal primeiro, depois envolve as variáveis com chaves. 3.5999... não é bug: float binário, formata com <code>:.2f</code> e segue.	
P4 · CONDIÇÕES	
<code>== != >= <=</code>	comparar VALOR (nunca <code>is / is not</code>)
<code>x not in ["D", "N"]</code>	"nem D nem N" (o idioma certo)
<code>x != "D" and x != "N"</code>	equivalente verboso (com <code>!=</code> o conector é AND)
<code>x >= A and x <= B</code>	"entre A e B (inclusive)": ≥ menor E ≤ maior

x != "D" or "N" é SEMPRE True ("N" sozinho é `truthy`). Cada lado do `and/or` = comparação completa. Enunciado: OU = `or`, E = `and`, literal. "Até X" INCLUI X (`<=`): borda errada = rubrica inteira (sem parcial).

P5 · LAÇOS: FOR vs WHILE	
<code>for t in range(3):</code>	# nº de voltas
<code>CONHECIDO antes for item in lista:</code>	# visita cada item da coleção
<code>while saldo >= 4:</code>	# repete ENQUANTO condição valer
<code>saldo -= 4</code>	# algo muda a cada volta

⚠ Justificativa pronta: quantidade fixa antes do laço = `for` · "enquanto tiver/der" = `while`. "R\$40 dá 10 tentativas" é isca: saldo é variável, parada é condição. Bloco fecha DESIDENTANDO (`return` é de função, não fecha laço). Dentro do laço o item é `i`, nunca a lista (`split` é na fruta, não na cesta).

P6 · ESQUELETO: VALIDAÇÃO + FAIXAS	
<code>d = input("Dist: ") uf = input("UF: ").upper() if not d.isdigit() or uf not in ["SP", "PR"]: print("ERRO DE ENTRADA") # valida ANTES do <code>int()</code> else: d = int(d) if d <= 2000: # "até X" inclui X taxa = 5.0 elif d <= 8000: # <code>elif</code> = faixa seguinte taxa = d/1000 * 1.2 # 1.2 PONTO! (1,2 = tupla) else: taxa = 25.0 if uf == "PR": taxa *= 2 # dobro vale p/ fixo tb print(f"R\$ {taxa:.2f}")</code>	

P7 · ESQUELETO: ACUMULADOR (Q5)	
<code>cont = 0; total = 0</code>	# zera FORA
<code>for item in lista:</code>	# item = 1
<code>string</code>	
<code>partes = item.split(" ")</code>	
<code>valor = float(partes[2])</code>	# converte E atribui
<code>cont += 1</code>	
<code>total += valor</code>	
<code>print(f"... {partes[0]} ...")</code>	# 1 por item
<code>print(f"{cont} x - Total: R\$ {total:.2f}")</code>	
<code># FORA</code>	

P8 · ESQUELETO: DATA C/ CAMPO OPCIONAL	
<code>meses =</code>	
<code>['janeiro', 'fevereiro', 'março', 'abril',</code>	
<code>'maio', 'junho', 'julho', 'agosto', 'setembro',</code>	
<code>'outubro', 'novembro', 'dezembro']</code>	
<code>for reg in datas:</code>	
<code>partes = reg.split(" ")</code>	#
<code>grosso 1º</code>	
<code>dia, mes, ano = partes[0].split("/")</code>	
<code>txt = f"{int(dia)} de</code>	
<code>{meses[int(mes)-1]} de {ano}"</code>	
<code>if len(partes) == 2:</code>	#
<code>guarda!</code>	
<code>h, m = partes[1].split(":")</code>	
<code>h = int(h); m = int(m)</code>	
<code>txt += " à 1 hora" if h==1 else f"</code>	
<code>às {h} horas"</code>	
<code>if m == 1: txt += " e 1 minuto"</code>	
<code>elif m > 1: txt += f" e {m}</code>	
<code>minutos"</code>	
<code>print(txt)</code>	#
<code>m==0: omit</code>	
<code># data junta: ano=s[0:4] mes=s[4:6]</code>	
<code>dia=s[6:8]</code>	

⚡ PROTOCOLO (10 passos)	
0. Q0 primeiro	nome, turma AE_, código; não apagar os #
1. Triagem 60s	cada questão: conceito / ler pandas / if+validação / DataFrame / parse em laço / análise
2. Ordem	Q1→Q2 → Q5 (3pts) → Q3 → Q4. Travou 10min: pula e volta
3. Antes do código	<code>df.columns + head()</code> : nomes e valores REAIS
4. Consertos em LOTE	achou 3 problemas? conserta os 3, DEPOIS roda
5. Teste de fechamento	exemplo do enunciado + borda exata de cada faixa + caso de ERRO + UMA entrada de CADA categoria/estado
6. Warnings	<code>SyntaxWarning</code> DIZ o conserto; <code>FutureWarning</code> : ignora
7. Saída completa	pediu 3 valores? imprime os 3 (faltar 1 = rubrica zero)
8. Formato exato	copia símbolo do enunciado · <code>:.2f</code> · linha em branco do modelo
9. Run All	Restart & Run All sem erro (prosa em célula de código = TODA linha com #)
10. Varredura final	NENHUMA resposta em branco · Q0 preenchida

🔍 VERBO DA QUESTÃO → FERRAMENTA	
"só os que..." (filtrar)	df[df["col"] == x]
"troque/reajuste os que..."	df.loc[mask, "col"] = / +=
"média/total POR grupo"	groupby
"grade de X por Y"	pivot_table
"cruze as duas tabelas"	pd.merge(a, b, on=[...])
"quantos de cada"	value_counts()
"existe tal valor?"	x in df["col"].values
"a linha do id X"	df[df["id"]==x].iloc[0]

D0 · O MODELO: 2 OBJETOS + MÁSCARA	
DataFrame	a TABELA inteira (linhas × colunas)
Series	UMA coluna (ou linha extraída via .iloc[0], ou resultado de groupby)
MÁSCARA = coluna de True/False de uma comparação df["UF"] df["UF"] == "SP" SP True df[mask] mantém RJ False as True (= filtro SP True do Excel)	

⚠ Pergunta-reflexo antes de cada linha: "o que tenho na mão: tabela, coluna ou valor?" · "truth value of a Series is ambiguous" = coluna INTEIRA onde ia UM valor → usa linha["col"] ou o i do loop.

D1 · CRIAR / LER	
df = pd.read_excel("a.xlsx", sheet_name="planilha Excel")	
pd.read_excel("a.xlsx", index_col=0)	1ª coluna vira índice
pd.read_csv("a.csv")	CSV padrão (vírgula separa, ponto decimal)
pd.read_csv("a.csv", sep=";", decimal=",")	CSV BRASILEIRO: sem isso, nº vira texto
df = pd.DataFrame({"time": ["Palmeiras", "Flamengo"], "pontos": [68, 68]}) # dict: chave=coluna	

⚠ Teste pós-leitura: df.dtypes → valor tem que ser float64/int64; object = virou texto, faltou decimal=", ". Confere o ARQUIVO certo da questão (MrPacoca ≠ pedidos).

D2 · EXPLORAR	
df.head() / df.tail()	5 primeiras / últimas linhas (DF INTEIRO, todas col.)
df.shape	(linhas, colunas), sem parênteses
list(df.columns)	nomes EXATOS (acento conta; errado = KeyError)
df.dtypes / df.info()	tipos / resumo
df["col"].unique()	QUAIS valores existem ("Pote", não "Potes")

D7 · ESQUELETO: REAJUSTE + DIFERENÇA	
total_pre = df["Total"].sum() # guarda ANTES df["Preço Unitário"] = df["Total"]/df["Quantidade"] df.loc[df["Mês"]==mes, "Preço Unitário"] += reajuste df["Total"] = df["Preço Unitário"]*df["Quantidade"] print(f'R\$ {df["Total"].sum() - total_pre:.2f}') # usa as VARIÁVEIS dadas, nunca os nº do exemplo	

D3 · FILTRAR / ALTERAR / 1 LINHA	
df["col"]	uma coluna (Series)
df["col"] == "SP"	MÁSCARA: True/False por linha
df[df["col"] == "SP"]	só as linhas True (tabela filtrada)
df.loc[mask, "col"] = v	REGRA DE OURO p/ trocar valor filtrado
df.loc[mask, "col"] += 0.04	somar só nos filtrados
df["nova"] = df["a"] * df["b"]	criar coluna (vetorizado, sem laço)
linha = df[df["id"]==x].iloc[0]	UMA linha como ficha → linha["col"]
x in df["col"].values	existe? SEM .values checa o ÍNDICE
df["col"].isin(["A", "B"])	máscara: está na lista?
df["col"].astype(str)	converter coluna (nº ↔ texto p/ comparar)
df["col"].str[0:3] / .str.upper()	.str = métodos de string na coluna inteira (fatia SÓ em colchetes)

TABELA × LINHA × ITEM (3 crashes): "truth value of a Series is ambiguous" = coluna inteira dentro de if. Depois do .iloc[0], TUDO vem de linha[...]; dentro de loop, de i. df = todo mundo · linha = um registro.

DIREÇÃO: reajuste/aumento = += · desconto = -= · |diferença| pode coincidir e esconder o erro: confere head() após a troca. NUNCA df[df[...]==x]["col"] = v (cópia). FutureWarning/Chained: IGNORAR (enunciado manda).

D3b · ANATOMIA DO .loc	
df.loc[MÁSCARA, "coluna"]	L quais LINHAS L qual COLUMNA
df.loc[df["UF"]=="SP", "Total"]	LER: a coluna, só nas linhas filtradas
df.loc[df["UF"]=="SP", "Preço"] = 9	ESCREVER nos filtrados (+= soma, *= mult.)
df.loc[mask]	sem vírgula: TODAS as colunas das linhas True
(m1) & (m2) / (m1) (m2)	E / OU entre máscaras (cada uma entre parênteses)
.loc vs .iloc[0]	loc = por nome/máscara · iloc = por POSIÇÃO (0 = primeira)

A máscara tem a COLUMNA à esquerda: df["UF"] == "SP". Escrever "UF" == "SP" compara dois textos, vale False, e o .loc procura uma linha CHAMADA False → KeyError: False. Proibidas: df[mask]["col"] = v (cópia, não altera) · df[mask, "col"] (sem .loc, inválido) · método sem () não executa (.min ≠ .min()).

D4 · AGREGAR / CONTAR	
df["col"].sum() .mean() .min() .max()	soma, média, mín, máx
df["col"].agg(["min", "max", "round(2)"])	vários stats em UMA operação (exigência típica)
df["col"].value_counts()	contagem POR categoria (NÃO fazer loop c/ contadores!)
df["col"].nunique()	QUANTAS categorias distintas
mask.sum()	conta True (True=1). .count() = não-nulos, NÃO use
serie.sort_values(ascending=False)	decrecente; .head(5) = top 5
df["col"].describe()	pacote completo: count, mean, std, min, quartis, max
df.sort_values("col", ascending=False)	ordena a TABELA inteira por uma coluna

D5 · ANÁLISE (Excel → pandas)	
df.groupby("g")["v"].mean()	Tabela Dinâmica de 1 medida = ranking por grupo
df.groupby("g")["v"].agg(["mean", "max"])	vários stats por grupo
pd.merge(a, b, on=["k1", "k2"])	PROCV por N chaves; chave faltando MULTIPLICA linhas (teste: shape!)
df.pivot_table(index="Estado", columns="Produto", values="Total", aggfunc="mean").reset_index()	Tabela Dinâmica: index=Linhas, columns=Colunas, values=Valores, aggfunc=Resumir por df.columns.name=None limpa rótulo
(df["m"] < 0).replace({True:"prejuízo", False:"lucro"})	classificar sem if
np.where(cond, "sim", "não")	idem (import numpy as np)

groupby = SEPARA → APLICA → JUNTA
SP→[.05,.07]→.06 | RJ→[.04]→.04 (grupo no índice)
2 chaves = 1 linha por COMBINAÇÃO existente
reset_index: rótulo da margem → coluna comum
MG ETANOL|.09 → 0|MG|ETANOL|.09 (margem vira 0,1,2)

⚠ TRIAGEM: dataset grande + média/ranking/cruzar = pandas SEM for. Lista de strings item a item = for + split (lado PYTHON). Ranking liderado por grupo de amostra MÍNIMA = liderança frágil: olha o value_counts() antes de confiar.

D6 · ESQUELETO: BUSCA ID + ÁRVORE	
pid = int(input("id: ")) # int: coluna é nº! if pid not in df["id"].values: print("Paciente não encontrado") else: linha = df[df["id"]==pid].iloc[0] imc = linha["peso"]/linha["altura"]**2 if linha["sist"] >= 140 or linha["diast"] >= 90: r = "consulta imediata" # regra 1 GANHA elif imc >= 30 and linha["fum"]=="sim": r = "risco alto" # ordem elif = prioridade elif imc >= 30 or linha["idade"] >= 60: r = "risco moderado" # OU do enunciado = or else: r = "risco baixo" print(f"Paciente {pid} (IMC: {imc:.1f}) - Risco: {r}") # travessão - : COPIA do enunciado	